

BittyVault Mainnet Etherscan Guide

Generated from the mainnet/.claude/commands skill set on 2026-06-25.

This guide explains how to create and manage a BittyVault on Ethereum mainnet from Etherscan. Mainnet transactions use real funds. Test every workflow on Sepolia first, verify every address, and confirm the connected wallet before signing.

1. How Etherscan Contract Tabs Work

Open a contract page on https://etherscan.io/address/<contract_address> and select the Contract tab.

- Read Contract: view data. No wallet connection is required.
- Write Contract: submit transactions. Click Connect to Web3, connect the correct wallet, fill the function fields, and confirm in your wallet.
- Read as Proxy / Write as Proxy: use these tabs if Etherscan shows the vault as a proxy and exposes proxy-specific interaction tabs.

Etherscan's own guide says Read Contract is for querying contract data, while Write Contract requires connecting a wallet through Connect to Web3.

Source: <https://info.etherscan.com/how-to-use-read-or-write-contract-features-on-etherscan/>

2. Mainnet Addresses

Name	Address
BittyVaultFactory	0x0000000094B81677434600b69d739Bc62b66a9c3
WETH	0xC02aaA39b223FE8D0A0e5C4F27eAD9083C756Cc2
WBTC	0x2260FAC5E5542a773Aa44fBCfE7C193bc2C599
USDC	0xA0b86991c6218b36c1d19D4a2e9Eb0cE3606eB48
USDT	0xdAC17F958D2ee523a2206206994597C13D831ec7
USDS	0xdC035D45d973E3EC169d2276DDab16f1e407384F
Aave V3 protocol	0x6F8B36cd866f91F844446d16f9FA8dEA09AF6cF4
Lido V2 protocol	0x4115bB297f21247FC55FD6255f0F8800d4172AF7
Uniswap V3 protocol	0x3b9384Ea4db89Af8Af54489779333b5A9c2b0436
Sky V1 protocol reference	0x350758FA196c94aB4309CD4A953e0097cEAB7cF5
Uniswap V3 NonfungiblePositionManager	0xC36442b4a4522E871399CD717aBDD847Ab11FE88

Asset decimals:

Token	Decimals
WETH	18
WBTC	8
USDC	6
USDT	6
USDS	18

Role hash:

Role	Hash
ASSET_MANAGER_ROLE	0x7613a25ecc738585a232ad50a301178f12b3ba8d3c8deb6a0dfa0418b2964fce

3. Wallets and Roles

Use separate wallets for separate duties.

Wallet	Purpose
Owner wallet	Holds DEFAULT_ADMIN_ROLE. Use for name changes, asset/protocol allowlist changes, receiver setup, and granting/revoking asset managers. Prefer a hardware wallet or multisig.
Asset manager wallet	Holds ASSET_MANAGER_ROLE. Use for trading, lending, staking, liquidity, and operational transactions. Keep enough ETH for gas.
Public caller	Some actions, such as payReceiver, may be callable by anyone unless receiver configuration restricts the trigger address.

Do not grant ASSET_MANAGER_ROLE to the same address as the owner if the vault contract rejects owner and asset manager equality.

4. Create a Vault on Etherscan

1. Open the factory page:
<https://etherscan.io/address/0x0000000094B81677434600b69d739Bc62b66a9c3#writeContract>
2. Click Connect to Web3.
3. Connect the deployer wallet. The mainnet skill uses the asset manager wallet as the transaction sender.
4. Find deployVaultAllSelected.
5. Fill the fields:

Field	Value
owner	Owner wallet address name Vault display name, or blank string assetManagers Asset manager wallet address as an array, for example [0xYourAssetManagerAddress]

deployVaultAllSelected uses the factory's selected/default mainnet asset and protocol configuration. Use deployVaultWithSelected only if you need to provide custom asset or protocol arrays manually.

6. Submit the transaction and wait for confirmation.
7. Copy the transaction hash and open it on Etherscan.
8. Compute or find the vault address:
 - On the factory Read Contract tab, call computeVaultAddress(owner,name).
 - Use the exact owner address and exact vault name used in deployVaultAllSelected.
 - Open https://etherscan.io/address/<vault_address>.

5. Verify a New Vault

On the vault contract page, use Read Contract:

Function	Expected result
name()	The vault display name.
getAssets()	WETH and WBTC by default.
getStableCoins()	USDC, USDT, and USDS by default.
getLendingProtocols()	Aave V3 protocol address.
getStakingProtocols()	Lido V2 protocol address.
getAMMProtocols()	Uniswap V3 protocol address.
hasRole(ASSET_MANAGER_ROLE, asset_manager)	true for the chosen asset manager.

For wallet balances, open each token contract, use Read Contract, and call balanceOf(vault_address). Convert raw units by token decimals.

6. Fund the Vault

The mainnet skill funds with WETH:

1. From an ETH-funded wallet, open WETH:
<https://etherscan.io/address/0xC02aaA39b223FE8D0A0e5C4F27eAD9083C756Cc2#writeContract>
2. Connect the wallet.
3. Call deposit() and enter the ETH amount in the transaction value field if Etherscan exposes a payable value input.
4. After wrapping, call WETH transfer(vault_address, amount_raw).
5. Verify by calling WETH balanceOf(vault_address).

Raw amount examples:

	Human amount	Token	Raw amount
	---	---	---
	0.01	WETH	1000000000000000000
	1	WETH	10000000000000000000
	100	USDC	100000000
	100	USDT	100000000
	0.01	WBTC	1000000

7. Owner Management Actions

Use the owner wallet on the vault Write Contract tab.

Action	Function	Inputs
---	---	---
Set name	setName	New display name.
Add assets	addAssets	Array of token addresses. Check isAddingAssetsDisabled() first.
Remove assets	removeAssets	Array of token addresses. Check current getAssets() and getStableCoins() first.
Lock assets	disableAddingAssets	No inputs. Irreversible.
Add lending protocols	addLendingProtocols	Array of protocol addresses.
Add staking protocols	addStakingProtocols	Array of protocol addresses.
Add AMM protocols	addAMMProtocols	Array of protocol addresses.
Remove protocols	matching remove function	Array of protocol addresses.
Lock protocols	disableAddingProtocols	No inputs. Irreversible.
Grant asset manager	grantRole	Role hash and address.
Revoke asset manager	revokeRole	Role hash and address.
Set rebalance limits	setRebalanceConfig	Asset, minimum balance, minimum duration, maximum amount.
Set receiver protection	setReceiverProtectionDuration	Duration in seconds.

Before writing, use Read Contract to inspect the relevant current state. After writing, refresh the same read function and confirm the transaction on Etherscan.

8. Receiver Payments

Use the owner wallet for receiver setup.

addReceiver fields from the mainnet skill:

Field	Meaning
---	---
name	Receiver identifier.
receiver_address	Payment destination.
asset	Token symbol or token address.
amount	Human amount converted to raw units.
payment_count	Number of payments.
start_timestamp	Unix timestamp when payments begin.
duration_seconds	Payment interval or total duration as expected by the vault ABI.

trigger	Optional trigger address. Use 0x00000000000000000000000000000000 if unrestricted.
immutable	Whether receiver config can be changed.
pay_insufficient	Whether to pay available balance if full amount is unavailable.

To pay a receiver, use `payReceiver(name, amount_raw)` if the vault exposes that function. A public caller can submit the transaction unless the receiver has a trigger restriction.

9. Asset Manager Operations

Use the asset manager wallet on the vault Write Contract tab.

Operation	Function	Mainnet protocol
Swap	rebalance	Uniswap V3: 0x3b9384Ea4db89Af8Af54489779333b5A9c2b0436
Supply	supply	Aave V3: 0x6F8B36cd866f91F844446d16f9FA8dEA09AF6cF4
Withdraw	withdraw	Aave V3: 0x6F8B36cd866f91F844446d16f9FA8dEA09AF6cF4
Stake	stake	Lido V2: 0x4115bB297f21247FC55FD6255f0F8800d4172AF7
Request unstake	unstake	Lido V2: 0x4115bB297f21247FC55FD6255f0F8800d4172AF7
Claim unstaked	claimUnstaked	Lido V2: 0x4115bB297f21247FC55FD6255f0F8800d4172AF7
Add liquidity	addLiquidity	Uniswap V3 protocol adapter
Remove liquidity	removeLiquidity	Uniswap V3 protocol adapter
Claim fees	claimFees	Uniswap V3 protocol adapter

For swaps and Uniswap liquidity operations, Etherscan may require encoded bytes data. The mainnet skills build those bytes with Foundry cast. Generate the encoded payload locally, then paste the bytes into Etherscan.

Swap data shape used by the skill:

```
f(address,uint256,address,uint256,bytes)
tokenIn, amountInRaw, tokenOut, amountOutMinRaw, path
```

Uniswap V3 path format:

0x + tokenIn_without_0x + fee_as_3_byte_hex + tokenOut_without_0x

Common fee tiers:

Fee tier	Percent
100	0.01%
500	0.05%
3000	0.30%
10000	1.00%

10. Operational Checklist

Before every mainnet write:

- Confirm the browser is on etherscan.io, not a testnet explorer or lookalike.
- Confirm the connected wallet is the correct owner or asset manager wallet.
- Confirm the contract address is the factory or the intended vault.
- Confirm raw token amounts and decimals.
- Run the matching Read Contract check first.
- Simulate or dry-run with tooling when the action has complex bytes or meaningful slippage risk.
- Keep the Etherscan transaction link in the operations log.

After every write:

- Wait for the transaction to succeed.
- Re-run the relevant Read Contract function.
- Check token balances with `balanceOf(vault_address)`.
- For role changes, call `hasRole(role_hash, address)`.
- For Uniswap V3 positions, save the NFT token ID from transaction logs.